

DD: BIL-02-GEN-01 — Invoice Generation

Developer implementation guide: DD_BIL-02-GEN-01-Invoice_Generation-DEV_IMPL_GUIDE-v1.3.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract
DD_BIL-02-GEN-01-Invoice_Generation-v1.3.md.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing & Revenue | **Version:** v1.3 Satellite of **BIL-02 Invoicing (Core)**. Owns the generator classes that produce invoices and pro forma documents in response to triggers (cycle close events, one-off charge requests, scheduled scanners, admin actions). Also owns idempotency, bulk reversal, and the generation failure queue.
Sibling DDs: BIL-02-TAX-01 Tax Invoice & Gateway, BIL-02-ADJ-01 Invoice Adjustments, BIL-02-READ-01 Invoice Read API, BIL-02-STA-01 Status Management & Recovery. **Version history:** see
CHANGELOG_BIL-02-GEN-01-Invoice_Generation.md in the same directory.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/05_billing/DD_BIL-02-GEN-01-Invoice_Generation-v1.3.md
Version	v1.3
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- GET /api/bulk-reversals/preview?...
- GET /api/bulk-reversals/{id}
- GET /api/bulk-reversals/{id}/failures
- GET /api/bulk-reversals?operator={code}&status={s}
- GET /api/generation-failures
- GET /api/generation-failures/{id}

- GET /api/invoices/{id}
- GET /api/invoices/{id}/pdf
- GET /api/invoices/{id}/status
- POST /api/billing/charges/compute
- POST /api/bulk-reversals
- POST /api/bulk-reversals/{id}/approve
- POST /api/bulk-reversals/{id}/cancel
- POST /api/credit-notes
- POST /api/generation-failures/{id}/resolve-no-action
- POST /api/generation-failures/{id}/retry
- POST /api/invoices/cycle/generate
- POST /api/invoices/manual
- POST /api/invoices/oneoff
- POST /api/invoices/{id}/cancel

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- billing-engine
- billing-invoicing
- billing-invoicing-gen
- put-master-fields

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- InvoiceCancelled
- InvoiceGenerationContext
- InvoiceGenerator
- InvoiceGeneratorRegistry
- InvoiceIssued
- InvoiceWriter

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing & Revenue | **Version:** v1.3

Satellite of **BIL-02 Invoicing (Core)**. Owns the generator classes that produce invoices and pro forma documents in response to triggers (cycle close events, one-off charge requests, scheduled scanners, admin actions). Also owns idempotency, bulk reversal, and the generation failure queue.

Sibling DDs: BIL-02-TAX-01 Tax Invoice & Gateway, BIL-02-ADJ-01 Invoice Adjustments, BIL-02-READ-01 Invoice Read API, BIL-02-STA-01 Status Management & Recovery.

Version history: see `CHANGELOG_BIL-02-GEN-01-Invoice_Generation.md` in the same directory.

Glossary

Term	Meaning
Generator class	A class that knows how to produce one specific type of invoice or pro forma document. Each invoice type (CYCLE_POSTPAID, ONE_OFF_INVOICE, PRO_FORMA_CYCLE, CREDIT_NOTE, DEBIT_NOTE) has its own generator class. Tax invoice generation lives in BIL-02-TAX-01, not here.
Trigger	The event or action that causes the system to invoke a generator: a Kafka event (cycle close, one-off charge requested), a scheduled scanner tick (daily pro forma run), or an admin action (manual re-issue from admin console).
Idempotency	The property that running the same generator with the same inputs more than once produces the same single output, not duplicates. Critical for Kafka consumer retries — if the consumer crashes after writing the invoice but before acknowledging the event, the retry must NOT create a second invoice.
Bulk reversal	An operator-initiated operation that cancels many invoices at once (entire billing run gone wrong) and optionally re-issues them with corrected values. Authorized via dual approval (Billing Admin + Finance Head).
Generation failure queue	A table that captures generation attempts that failed in a recoverable way (BIL-01 unavailable, tax service timeout, customer master fetch failed). A scheduled retry job picks up entries and retries them; admin can manually resolve or give up.
MACD	Move / Add / Change / Disconnect — workflow operations that may carry a one-off fee. Examples: address relocation fee, equipment swap fee, mid-cycle package change fee. These fees flow as OneOffChargeRequested events from the subscription workflow modules to GEN-01.
Charge	A pre-tax line item returned by BIL-01 Charging Engine for a given subscription + intent + cycle window. Each charge carries a wallet_type_code and a package_ref so GEN-01 can apply the operator's grouping policy.
Line item builder	A shared service in GEN-01 that takes BIL-01 charges + per-line tax breakdowns from PLM-CFG-02 Tax Configuration and produces the two-level (SUMMARY + DETAIL) invoice_line_item rows that go on the invoice.
Pro forma scanner	A scheduled job that runs once per day per operator and generates pro forma documents for PREPAID subscriptions whose next cycle is within the pre-cycle window (default 5 days).

A — Target

1. What this process is about

GEN-01 owns **producing invoices and pro forma documents**. The triggers come from elsewhere: BIL-03 Cycle Close emits a `CycleClosed` event when a subscription's cycle window has elapsed; the subscription workflow modules emit `OneOffChargeRequested` events when a workflow incurs a one-off fee (relocation, equipment swap, package change); a scheduled scanner wakes up daily to generate pro forma for upcoming PREPAID cycles; BIL-02-ADJ-01 Invoice Adjustments calls credit/debit note generators after approving an adjustment.

For each trigger, GEN-01 instantiates the right generator class via the framework in BIL-02 Invoicing (Core), passes a generation context, and writes one or more invoices (one per group in the operator's grouping policy). The generator does the orchestration work: subscription state checks, charge computation calls to BIL-01 Charging Engine, tax computation calls to PLM-CFG-02 Tax Configuration, customer snapshot capture, line item building, sequence allocation, and event emission.

GEN-01 also owns operational features tied to generation: idempotency on every retry path, bulk reversal/re-bill when whole batches need correction, and a generation failure queue for recoverable errors.

Five generators owned by GEN-01

Generator	Invoice type produced	Trigger source
<code>CyclePostpaidInvoiceGenerator</code>	CYCLE_POSTPAID	Kafka event <code>sophix.billing.cycle.closed</code> from BIL-03 Cycle Close, filtered for POSTPAID subscriptions
<code>OneOffInvoiceGenerator</code>	ONE_OFF_INVOICE	Kafka event <code>sophix.billing.charge.oneoff.requested</code> from subscription workflows or admin
<code>ProFormaCycleGenerator</code>	PRO_FORMA_CYCLE (pro forma table)	Scheduled scanner at 03:00 local-time per operator
<code>CreditNoteGenerator</code>	CREDIT_NOTE	Called by BIL-02-ADJ-01 Invoice Adjustments after approving a credit adjustment
<code>DebitNoteGenerator</code>	DEBIT_NOTE	Called by BIL-02-ADJ-01 Invoice Adjustments after approving a debit adjustment

What GEN-01 explicitly does not own

- **Tax invoice generation** — that's a separate generator class living in BIL-02-TAX-01 Tax Invoice & Gateway, with its own trigger (payment applied event), its own signing flow, and its own retry framework.
- **Adjustment approval workflow** — BIL-02-ADJ-01 Invoice Adjustments owns the capture, threshold routing, and approval. GEN-01 only owns the CreditNoteGenerator and DebitNoteGenerator classes that ADJ-01 calls.
- **Invoice lifecycle transitions after generation** — once an invoice is GENERATED, transitions to PENDING / PAID / OVERDUE / CANCELLED are owned by BIL-02-STA-01 Status Management & Recovery (and BIL-02 Core's lifecycle state machine).
- **PDF rendering or delivery** — DD_NOT-01 Notification Service renders PDFs from the events GEN-01 emits; BIL-02-READ-01 Invoice Read API owns the retrieval API.
- **Charging logic** — BIL-01 Charging Engine owns what to charge; GEN-01 just calls it.
- **Tax computation rules** — PLM-CFG-02 Tax Configuration owns rule logic; GEN-01 calls TaxService and stores results.

2. Business rules and generator flows

This section is organized to give a dev a complete view of each generator in one place: trigger → rules → step-by-step flow → output → failure paths.

Rules split into:

- **Shared rules** (F = framework, L = line builder, Q = failure queue, R = bulk reversal) — apply to multiple/all generators
- **Per-generator sections** — five subsections, one per generator (cycle, one-off, pro forma, credit note, debit note), each with its specific rules and an end-to-end walkthrough

Shared rule group F — Generator framework (applies to ALL generators)

ID	Rule	Triggered on
R-GEN-01-F-1	All generators implement the InvoiceGenerator interface defined in BIL-02 Invoicing (Core). Each generator is a Spring @Component (Java) or service-container binding (Laravel) scanned at module startup. The InvoiceGeneratorRegistry indexes generators by getTypeCode() for runtime dispatch.	Generator registration
R-GEN-01-F-2	Each trigger source (Kafka consumer, scheduled scanner, admin endpoint, ADJ-01 callout) is responsible for building the InvoiceGenerationContext and calling InvoiceGenerationService.generate(context). The service dispatches to the registered generator via the registry. Trigger sources never call generators directly.	Dispatch through service
R-GEN-01-F-3	A generator returns a List<Invoice> (possibly empty). Empty means the trigger resulted in no invoices — zero-charge cycles, PENDING subscription state, etc. Multiple invoices result from the operator's grouping policy producing more than one group of charges.	Generator return contract
R-GEN-01-F-4	Every generator must perform the PENDING subscription state check before generating anything. If the subscription is in any PENDING_* state (mid-workflow per SUB-LM-01 Subscription Lifecycle Management), generation is deferred — the generator enqueues a retry entry in the generation failure queue with reason_code = SUBSCRIPTION_PENDING and returns an empty list. The retry scanner picks it up after the subscription leaves PENDING.	PENDING state gate

ID	Rule	Triggered on
R-GEN-01-F-5	Every generator must check idempotency BEFORE writing. The check key depends on the document type: (triggering_event_ref, invoice_type_code, grouping_config_ref) for invoice documents; (subscription_id, idempotency_cycle_key, grouping_config_ref) for pro forma. If a matching record exists, the generator returns it instead of writing a new one. This makes Kafka retries safe and admin re-inocations safe.	Idempotency check
R-GEN-01-F-6	Every invoice generator captures a customer_snapshot JSONB at the moment of generation via CustomerSnapshotService (in Core). The snapshot is stored on the invoice / pro forma row and never modified thereafter. If the customer service is unavailable, the generator enqueues a retry entry with reason_code = CUSTOMER_SNAPSHOT_FETCH_FAILED — the invoice is NOT written with a partial snapshot.	Customer snapshot capture
R-GEN-01-F-7	All invoice writes happen in a single DB transaction via Core's InvoiceWriter (for invoices) or ProFormaWriter (for pro forma). The transaction includes: sequence number allocation, invoice row insert, line item inserts, and emission of the InvoiceIssued or ProFormaIssued event to the outbox. If any step fails, the entire transaction rolls back AND a row is recorded in invoice_sequence_gap (for legal documents) so the allocated number is audited as deliberately skipped.	Atomic write

Shared rule group L — Line item construction (used by all generators)

ID	Rule	Triggered on
R-GEN-01-L-1	The line item builder is invoked by every generator. It takes the BIL-01 charges (or adjustment line items, for credit/debit) and produces the two-level (SUMMARY + DETAIL) invoice_line_item rows that go on the invoice. Each SUMMARY line represents one chargeable thing the customer sees on the summary page; each DETAIL line is a sub-breakdown attached to its parent SUMMARY line via parent_summary_line_id.	Two-level structure
R-GEN-01-L-2	For each charge from BIL-01, the builder calls PLM-CFG-02 Tax Configuration's TaxService.computeTax(charge, customerLocation, packageRef) to get the per-line tax_breakdown JSONB. The breakdown is stored on the line as-is. The builder then aggregates per-line breakdowns into the invoice's tax_summary JSONB (totals per tax component across all lines).	Tax application
R-GEN-01-L-3	The builder respects the line's wallet_type_code (and, for non-WALLET dimensions, package_ref or service_category_code). When the generator filters charges for a specific group, only matching lines are passed to the builder — so all lines on the resulting invoice naturally belong to that group.	Group-respecting
R-GEN-01-L-4	Line item ordering on the invoice is controlled by sort_order (integer). SUMMARY lines get values 10, 20, 30, ...; their DETAIL children get 11, 12, 13 (under SUMMARY 10), 21, 22, 23 (under SUMMARY 20), etc. The default ordering follows the order of charges returned by BIL-01; operators can configure custom ordering via PLM-CFG-01 Package Catalog metadata if needed.	Ordering

ID	Rule	Triggered on
R-GEN-01-L-5	Line item descriptions are localized to the customer's language preference (from the customer snapshot). For each charge, BIL-01 returns a translation key; the builder resolves it via the operator's translation catalog. If a translation is missing for the customer's preferred language, the operator's default language is used as fallback. The translated string is stored on the line at write time (not re-translated at render time) — same immutability principle as customer snapshot.	Translation

Shared rule group Q — Generation failure queue (used by all generators)

Invoice generation can fail partway through for reasons that are temporary rather than permanent — a downstream service is briefly unreachable, the customer master has a stale lock, the subscription is mid-workflow and can't be billed yet. These failures are **recoverable**: if we retry in 15 minutes, the problem is usually gone.

Without a retry queue, the only options would be: (a) lose the invoice (the cycle close event was consumed and acknowledged, but no invoice exists for that customer that cycle — bad), or (b) keep retrying inline forever blocking the consumer thread (bad), or (c) fail loud and require manual intervention every time (bad at scale — a 5-minute BIL-01 outage could create 10,000 manual tickets).

The generation failure queue is the fourth option: persist the failure with full context (what was being generated, for which subscription, why it failed, what payload to retry with), let the Kafka consumer commit and move on, and have a separate scanner pick up the queue every 15 minutes to retry. Exponential backoff means transient issues self-heal without operator involvement; persistent issues eventually escalate to admin review.

Concrete scenarios this handles:

- A subscription is in PENDING_UPGRADE state when its cycle closes — can't bill until the upgrade workflow finishes. Queue with reason SUBSCRIPTION_PENDING; retry every 15 minutes; succeeds once workflow completes (usually within an hour).
- BIL-01 Charging Engine has a 20-minute outage during a cycle close batch — instead of losing 5,000 invoices, all 5,000 land in the queue; BIL-01 comes back; next scanner tick regenerates all of them successfully.
- PLM-CFG-02 Tax Configuration times out on a specific customer's tax computation due to an exemption rule edge case — queued; admin reviews after 8 retries (~3 days) and sees the underlying tax config bug that needs fixing.
- CRM is unreachable during pro forma generation — queued; pro forma is informational, so deferred generation has no customer impact; succeeds on retry.

The queue is admin-visible: after 8 retries (~3 days) the entry moves to GAVE_UP_AUTO for human review, and operators can monitor what's failing in real time (a spike of TAX_SERVICE_FAILED reasons in one hour is a tax service incident, not 200 isolated failures).

What the failure queue is NOT for:

- Permanent errors (malformed event payload, invalid subscription ID, code bug throwing IllegalStateException) — those go to the Kafka dead-letter topic for engineering, not the failure queue. The queue is only for failures where "retrying later" is a sensible response.
- Successful generation (the queue only has rows when something went wrong; successful generation leaves no trace here).

ID	Rule	Triggered on
R-GEN-01-Q-1	When a generator cannot proceed due to a recoverable error (PENDING subscription state, BIL-01 unavailable, PLM-CFG-02 timeout, customer snapshot fetch failed, sequence allocation failure rolled back, etc.), the generator enqueues a row in generation_failure_queue capturing: invoice type, triggering event ref, subscription_id, operator_code, reason_code, reason_detail (free-form), retry_count = 0, next_retry_at = now + initial backoff. The generator returns an empty list to its caller — no partial write.	Failure capture

ID	Rule	Triggered on
R-GEN-01-Q-2	A scheduled scanner runs every 15 minutes per operator scanning generation_failure_queue for rows where next_retry_at <= now() AND status = PENDING_RETRY AND retry_count < max_retries. For each row, it reconstructs the generation context (looking up subscription state, original event payload, etc.) and re-invokes the generator. On success: row status becomes RETRIED_SUCCESS. On failure: retry_count increments, next_retry_at extends via exponential backoff (15min, 30min, 1h, 2h, 4h, 8h, 24h, then daily).	Retry scanner
R-GEN-01-Q-3	After max retries (default 8 attempts spanning ~3 days), status becomes GAVE_UP_AUTO. An alert fires for operator review. Admin can examine the failure detail and either: (a) manually trigger another retry (status returns to PENDING_RETRY with retry_count reset); (b) mark as RESOLVED_NO_ACTION (operator decided not to generate this invoice — usually because the underlying business case is no longer valid); (c) escalate per operator policy.	Give up + admin action
R-GEN-01-Q-4	The generation failure queue is NOT used for unrecoverable errors (malformed event payload, invalid subscription ID, invalid invoice type). Those go to the standard Kafka dead-letter topic for engineering review and are NOT auto-retried.	Recoverable only

Shared rule group R — Bulk reversal (operator-driven, affects already-generated invoices)

Sometimes an entire batch of invoices needs to be cancelled and re-issued because they were all generated with a systemic error — the wrong tax rate, the wrong package price, the wrong grouping policy, a buggy charging computation. The invoices already exist in the system. Many of them have already been sent to customers by email. Some may have been notified to the tax authority. The operator needs a clean, auditable way to cancel them all and (optionally) re-issue corrected versions, without writing one-off SQL scripts or clicking through admin UI 10,000 times.

That's bulk reversal. It is **always initiated by a human** — a billing admin in the operator's admin console. It is never triggered by an automatic system event. The mechanism: define a scope (operator + invoice type + date range + optional filters), preview the impact (how many invoices, what total amount), get dual approval (Billing Admin proposes, Finance Head approves), execute the reversal one invoice at a time in transactional batches with full audit trail, optionally re-invoke the generators to produce corrected replacements.

Concrete scenarios that require bulk reversal:

- An operator's tax team realizes that VAT was applied at 16% when it should have been 15% for September cycle invoices in Wananchi KE — 12,000 invoices need to be cancelled and re-issued with the right rate.
- A package price catalog update was committed with a typo (KES 3,500 became KES 35,000 due to a missing decimal) — affected cycle invoices have totals 10x too high, customers complained, all need to be re-issued.
- The grouping policy was misconfigured for one operator and a batch of cycle invoices was issued with the wrong wallet grouping (Internet and TV invoiced separately when they should have been grouped) — re-issue with corrected policy.
- BIL-01 Charging Engine had a regression for 24 hours causing usage charges to be doubled for postpaid customers — entire 24-hour batch needs reversal and re-issue.

Why this needs to be a first-class capability, not just SQL or stored procedures:

- **Scale:** 10,000+ invoices in a single batch. Manual UI cancellation impractical.
- **Atomicity at invoice level:** each invoice's reversal must be its own transaction (cancel + outstanding balance update + event emission). A bulk SQL UPDATE can't do this.
- **Protected states:** signed tax invoices can't simply be cancelled — tax authority reconciliation required. Already-cancelled invoices shouldn't be re-cancelled. Invoices linked to credit/debit notes need to be reversed as a coherent group. SQL doesn't enforce this.
- **Dual approval audit:** Finance regulators expect to see "who initiated, who approved, when, why" for any mass financial adjustment. The bulk_reversal_batch table captures this.
- **Per-invoice failure visibility:** when 12,000 invoices are processed, some will fail (linked to credit notes, in protected state, etc.). The admin needs to see which ones failed and why, with the ability to resolve them individually.

- **Optional re-issue:** cancellation alone leaves the customers with no invoice; the operator usually wants corrected versions issued automatically. The batch can re-invoke the appropriate generator with fresh inputs (fresh tax computation, fresh charges from BIL-01).
- **Downstream events:** every cancellation emits `InvoiceCancelled` consumed by BIL-01 Charging Engine (outstanding balance update), BIL-04 Dunning Engine (stop dunning), DD_NOT-01 Notification Service (notify customer). Doing this via SQL would miss the event emission and break consumers.

What bulk reversal is NOT:

- Not for cancelling a single invoice — that's a routine operation owned by **BIL-02-STA-01 Status Management & Recovery** through standard admin UI.
- Not for correcting just the amount on an invoice — that's an adjustment, owned by **BIL-02-ADJ-01 Invoice Adjustments**, which issues a credit or debit note rather than cancelling the original.
- Not for handling individual customer disputes — that's STA-01 (dispute hold) and ADJ-01 (adjustment if dispute upheld).
- Not for cancelling pro forma documents — pro forma supersession is automatic when the next pro forma is generated. There's no bulk reversal of pro forma.

Source / who initiates: Always a Billing Admin user via the admin console REST endpoints listed later in this DD. The flow:

1. Billing Admin defines scope and clicks "Preview" — system computes how many invoices match, what the total amount affected is, which would be rejected due to protected state.
2. Billing Admin reviews preview, adds operator notes explaining why, clicks "Submit for approval" — batch row created with status `PENDING_APPROVAL`.
3. Finance Head receives notification (typically via operator email/Slack), reviews in the admin console, clicks "Approve" — batch transitions to `IN_PROGRESS`.
4. Worker thread executes the batch one invoice at a time; admin can monitor progress in real time.
5. On completion: `COMPLETED` if all in-scope invoices processed successfully; `PARTIALLY_FAILED` if some failed. Per-invoice failures viewable in the failure detail screen.

ID	Rule	Triggered on
R-GEN-01-R-1	A bulk reversal is initiated by an operator admin when an entire billing run produced incorrect invoices (e.g., wrong tax rate applied, wrong package price). The admin specifies a scope: operator, invoice type, date range, optional package/wallet/category filter. Authorization requires <code>BILLING_ADMIN</code> role AND a separate approval by <code>FINANCE_HEAD</code> . Both approvals are captured in <code>bulk_reversal_batch</code> with timestamps and user IDs.	Authorization
R-GEN-01-R-2	The bulk reversal scope is validated before execution: the admin sees a preview of how many invoices match the scope and the total amount affected. The admin confirms; the batch row is written with status = <code>IN_PROGRESS</code> .	Preview + confirm
R-GEN-01-R-3	Bulk reversal execution iterates per-invoice in transactional batches. For each in-scope invoice: (a) the invoice's status moves to <code>CANCELLED</code> with <code>cancel_reason_code = BULK_REVERSAL</code> and <code>cancel_batch_id = batch.id</code> ; (b) BIL-01 Charging Engine receives the <code>InvoiceCancelled</code> event and reverses outstanding-balance entries; (c) if the batch has <code>re_issue = true</code> , the generator is re-invoked with fresh charges (BIL-01 recomputes with the corrected inputs) and fresh tax (PLM-CFG-02 recomputes with current rules).	Execution
R-GEN-01-R-4	Bulk reversal is blocked for invoices in certain protected states: (a) signed tax invoices (BIL-02-TAX-01 owns those — tax-authority reconciliation required for any reversal); (b) invoices that are already <code>CANCELLED</code> ; (c) invoices that have associated credit/debit notes (reversing one without the other creates accounting chaos — admin must reverse them as a coherent group via a different workflow). The system rejects these from the scope at preview time with a count and reason.	Protected states

ID	Rule	Triggered on
R-GEN-01-R-5	Bulk reversal produces full observability: <code>bulk_reversal_batch</code> tracks <code>invoices_in_scope</code> , <code>invoices_cancelled</code> , <code>invoices_re_issued</code> , <code>invoices_failed</code> , <code>started_at</code> , <code>completed_at</code> , <code>status</code> . Per-invoice failures are logged in <code>bulk_reversal_failure</code> so the admin can review and retry individually. A <code>BulkReversalCompleted</code> Kafka event fires when the batch finishes (either fully or partially).	Observability

Generator 1 — CyclePostpaidInvoiceGenerator

Purpose: produce the regular cycle invoice for POSTPAID subscriptions at cycle close.

Trigger: Kafka consumer of `sophix.billing.cycle.closed` (emitted by BIL-03 Cycle Close). Sample event payload:

```
{
  "eventId": "cycle_close_01HZQX5K3M9P0V8YQ6",
  "eventType": "CycleClosed",
  "eventTimestamp": "2026-10-01T00:01:30Z",
  "subscriptionId": "sub_01HVR3T2W5K8",
  "operatorCode": "WANANCHI_KE",
  "billingMode": "POSTPAID",
  "cycleStart": "2026-09-01T00:00:00Z",
  "cycleEnd": "2026-09-30T23:59:59Z"
}
```

The consumer filters: only events with `billingMode = POSTPAID` are dispatched to this generator. PREPAID cycle settlement is handled by BIL-05 Wallet and Topup, not GEN-01.

Specific rules:

ID	Rule	Triggered on
R-GEN-01-C-1	Generator is invoked from a Kafka consumer of <code>sophix.billing.cycle.closed</code> . Consumer filters for <code>billing_mode = POSTPAID</code> . PREPAID is not handled here.	Trigger source
R-GEN-01-C-2	Generator calls BIL-01 Charging Engine at <code>POST /api/billing/charges/compute</code> with { <code>subscriptionId</code> , <code>intent: 'CYCLE_CLOSE'</code> , <code>cycleWindow</code> , <code>triggeringEventRef</code> }. BIL-01 returns the list of charges, each tagged with <code>wallet_type_code</code> and <code>package_ref</code> .	Charge computation
R-GEN-01-C-3	Generator looks up the operator's grouping policy in <code>invoice_grouping_config</code> for (<code>operator_code</code> , <code>CYCLE_POSTPAID</code>). For each group: filter BIL-01 charges by the group's <code>group_key_values</code> (based on dimension). Empty group → skip (no zero-amount invoice written).	Group filtering
R-GEN-01-C-4	For each non-empty group: build full invoice (line items, totals, tax_summary) and write atomically. <code>customer_snapshot</code> captured once and reused across groups (same subscription, same moment).	Per-group generation
R-GEN-01-C-5	Due date is <code>issue_date + invoice_type_operator_config.due_date_grace_days</code> . System default of 14 days applies if operator hasn't set a value.	Due date
R-GEN-01-C-6	<code>billing_mode = POSTPAID</code> , <code>triggering_event_ref = <BIL-03 cycle close event ID></code> , <code>period_start</code> and <code>period_end</code> from cycle window.	Header fields

End-to-end flow (one `CycleClosed` event → 0..N invoices):

- Step 1 – Receive `CycleClosed` event from Kafka
Consumer pulls event; filters `billingMode = POSTPAID`; builds `InvoiceGenerationContext` with `subscriptionId`, `cycleWindow`, `triggeringEventRef`, `operatorCode`.
- Step 2 – Subscription state gate (R-GEN-01-F-4)
Read subscription snapshot via SUB-LM-01.
If status is `PENDING_*` → enqueue `gen_failure_queue` (reason: `SUBSCRIPTION_PENDING`) → return empty.
If status is `TERMINATED` or `SUSPENDED` → log + return empty (no recovery).
- Step 3 – Capture customer snapshot ONCE (R-GEN-01-F-6)

Call CustomerSnapshotService.captureSnapshot(customerId, now()).
 Failure → enqueue gen_failure_queue (reason: CUSTOMER_SNAPSHOT_FETCH_FAILED) → return empty.

Step 4 – Fetch charges from BIL-01 (R-GEN-01-C-2)

POST /api/billing/charges/compute with { subscriptionId, intent: CYCLE_CLOSE, cycleWindow, triggeringEventRef }.
 Receive list of charges, each with: wallet_type_code, package_ref, service_category_code, amount, description_key, quantity.
 Empty list → no charges this cycle → log + return empty (no invoice).
 Failure (network, timeout, 5xx) → enqueue gen_failure_queue (reason: BIL01_UNAVAILABLE) → return empty.

Step 5 – Load grouping policy for (operatorCode, CYCLE_POSTPAID) (R-GEN-01-C-3)

Query invoice_grouping_config; receive 1..N group rows with dimension + group_key_values.

Step 6 – Per group, generate one invoice (R-GEN-01-C-4)

For each group in policy:

Step 6a – Idempotency check (R-GEN-01-F-5)

Query for existing invoice with (triggering_event_ref, CYCLE_POSTPAID, group.id).
 If found → add to result list, skip to next group.

Step 6b – Filter charges for this group

Match charges by dimension: WALLET → wallet_type_code in group_key_values; PACKAGE → package_ref;
 SERVICE_CATEGORY → service_category_code; SINGLE → all charges.
 Empty filtered set → skip group (no invoice for this group this cycle).

Step 6c – Build line items via InvoiceLineBuilder (R-GEN-01-L-1, L-2, L-5)

For each filtered charge:

- Resolve description from translation key + customer language preference (from snapshot)
- Call PLM-CFG-02 TaxService.computeTax(charge, snapshot.billingAddress, charge.packageRef)
- Build SUMMARY line and DETAIL lines per package
- Store tax_breakdown JSONB on each line

Aggregate per-line tax breakdowns into invoice-level tax_summary.

Step 6d – Build invoice header

Set invoice.subscriptionId, customerId, operatorCode, currency (from subscription snapshot),
 billing_mode = POSTPAID, grouping_config_ref = group.id, grouping_dimension, grouping_key_values,
 customer_snapshot (from step 3), triggering_event_ref, issue_date = now(),
 period_start/end from cycleWindow.
 Compute due_date = issue_date + due_date_grace_days (operator config).
 Set status = GENERATED, payment_status = UNPAID.
 Compute subtotal_amount, tax_amount_total, total_amount from lines.

Step 6e – Allocate sequence + write atomically (R-GEN-01-F-7)

In one DB transaction:

- InvoiceSequenceService.allocateLegalNumber(operator, fiscalYear, CYCLE_POSTPAID, format)
- invoice row insert
- invoice_line_item bulk insert
- outbox event row insert (InvoiceIssued with full payload incl. customer_snapshot)

Any failure → transaction rolls back AND insert invoice_sequence_gap row with the allocated number
 and reason_code = LINE_ITEM_INSERT_FAILED (or similar) → enqueue gen_failure_queue → continue to next group.

Step 6f – Append persisted invoice to result list.

Step 7 – Return result list of invoices to consumer

Consumer commits Kafka offset → done.

Output:

- 0..N rows in invoice table (one per non-empty group), each with its own legal invoice number
- 0..N corresponding invoice_line_item rows (typically 2-10 lines per invoice with SUMMARY + DETAIL structure)
- 0..N InvoiceIssued Kafka events (consumed by DD_NOT-01 Notification Service for PDF + delivery, by BIL-01 Charging Engine for outstanding balance tracking, by analytics)
- Idempotent: same event reprocessed → no duplicate invoices

Failure paths:

Where it fails	Lands in	Recovery
Subscription PENDING	generation_failure_queue (reason: SUBSCRIPTION_PENDING)	Retry scanner re-tries every 15 min until subscription leaves PENDING
CRM/Customer service down	generation_failure_queue (reason: CUSTOMER_SNAPSHOT_FETCH_FAILED)	Retry with exponential backoff up to 8 attempts
BIL-01 down or 5xx	generation_failure_queue (reason: BIL01_UNAVAILABLE)	Retry with exponential backoff
PLM-CFG-02 TaxService timeout	generation_failure_queue (reason: TAX_SERVICE_FAILED)	Retry with exponential backoff
DB transaction rollback (constraint violation, etc.)	invoice_sequence_gap row + generation_failure_queue	Retry; gap row stays as audit

Where it fails	Lands in	Recovery
Unrecoverable (malformed event, invalid subscriptionId)	Kafka dead-letter topic	Engineering review

Generator 2 — OneOffInvoiceGenerator

Purpose: produce one-off invoices for MACD fees (relocation, equipment swap, package change mid-cycle, install fee, etc.) and other ad-hoc charges.

Trigger: Kafka consumer of `sophix.billing.charge.oneoff.requested` (emitted by subscription workflows or admin actions). Sample event payload:

```
{
  "eventId": "oneoff_charge_01HZQA2B5C8D9E",
  "eventType": "OneOffChargeRequested",
  "eventTimestamp": "2026-10-12T14:33:21Z",
  "subscriptionId": "sub_01HVR3T2W5K8",
  "operatorCode": "WANANCHI_KE",
  "requestedBy": "workflow:SUB-WF-RELOCATION-01:wf_01HZQA1...",
  "feeDescriptor": {
    "feeTypeCode": "RELOCATION_FEE",
    "amount": 2500.00,
    "currency": "KES",
    "wallet_type_code": "WALLET_INTERNET",
    "package_ref": "PKG_INTERNET_100M",
    "description_key": "fee.relocation.standard",
    "payment_handling": "ISSUE_INVOICE"
  },
  "billingMode": "POSTPAID"
}
```

The consumer filters: only events with `payment_handling = ISSUE_INVOICE` are dispatched to this generator. Events with `payment_handling = WALLET_DEBIT` are consumed directly by BIL-05 Wallet and Topup; GEN-01 never sees them.

Specific rules:

ID	Rule	Triggered on
R-GEN-01-K-1	Generator consumes <code>sophix.billing.charge.oneoff.requested</code> from subscription workflow modules (relocation, upgrade, equipment swap, package change, etc.) and from admin actions. Event carries: <code>subscription_id</code> , <code>requested_by</code> , <code>feeDescriptor</code> (charge type, amount, wallet/package refs, <code>payment_handling</code>).	Trigger source
R-GEN-01-K-2	For POSTPAID subscriptions: generator produces a <code>ONE_OFF_INVOICE</code> document with its own legal invoice number, its own due date, the standard lifecycle. Independent of cycle invoice — next cycle invoice does NOT roll it up.	POSTPAID one-off
R-GEN-01-K-3	For PREPAID subscriptions: handling chosen by <code>payment_handling</code> field on the fee descriptor. <code>WALLET_DEBIT</code> → BIL-05 consumes, GEN-01 NOT invoked. <code>ISSUE_INVOICE</code> → GEN-01 generates one-off invoice with <code>billing_mode = PREPAID</code> . The choice is per-fee-type, configured by the workflow module that emits the event.	PREPAID one-off
R-GEN-01-K-4	When <code>ISSUE_INVOICE</code> is signaled for a PREPAID subscription, the resulting invoice means "this PREPAID customer must pay this fee separately, outside their normal cycle flow."	Path signaling
R-GEN-01-K-5	One-off invoices respect the operator's grouping policy for <code>ONE_OFF_INVOICE</code> , which may differ from <code>CYCLE_POSTPAID</code> . A common case: cycle invoices grouped by wallet but one-off invoices with <code>SINGLE</code> dimension.	Grouping policy
R-GEN-01-K-6	<code>triggering_event_ref = <OneOffChargeRequested event ID></code> for idempotency. Workflow retries of the fee-emission step do NOT produce duplicate invoices.	Idempotency

End-to-end flow (one OneOffChargeRequested → 0..N invoices):

Step 1 – Receive `OneOffChargeRequested` from Kafka

Consumer pulls event; filters payment_handling = ISSUE_INVOICE; builds InvoiceGenerationContext with subscriptionId, operatorCode, feeDescriptor (the charge details), triggeringEventRef.

Step 2 – Subscription state gate (R-GEN-01-F-4)

Same as cycle generator. PENDING_* → enqueue, return empty.

Step 3 – Capture customer snapshot (R-GEN-01-F-6)

Same as cycle generator.

Step 4 – Treat feeDescriptor as a single virtual "charge"

Unlike cycle invoice (which fetches charges from BIL-01), one-off invoice gets its single charge directly from the event payload. The feeDescriptor IS the charge.

No BIL-01 call needed for the charge itself – but BIL-01 IS notified after the invoice is issued (via the InvoiceIssued event consumer) so it can track outstanding balance.

Step 5 – Load grouping policy for (operatorCode, ONE_OFF_INVOICE) (R-GEN-01-K-5)

Typically SINGLE dimension (one invoice per fee), but operator may have configured otherwise.

Step 6 – For each group, generate one invoice

Almost always exactly one group (because there's only one charge to place).

Sub-steps identical to cycle generator's Step 6:

- Idempotency check on (triggering_event_ref, ONE_OFF_INVOICE, group.id)
- Match the single feeDescriptor against the group's keys → either it matches (one invoice) or it doesn't (skip)
- Build line items via InvoiceLineBuilder:
 - * One SUMMARY line for the fee
 - * Optionally one DETAIL line if the operator wants the fee broken down (rare for one-offs)
 - * Tax via PLM-CFG-02 TaxService on the fee amount
- Build header: billing_mode from event (POSTPAID or PREPAID), grouping_config_ref, customer_snapshot, triggering_event_ref = OneOffChargeRequested event ID, issue_date = now(), due_date = now() + grace_days (operator config; one-off may have different grace than cycle), period_start/end = NULL (one-off not tied to a cycle window)
- Allocate sequence (ONE_OFF_INVOICE type has its own gap-free sequence)
- Write atomically

Step 7 – Return result list to consumer

Consumer commits Kafka offset → done.

Output:

- Typically 1 invoice (one fee → one invoice). Rare cases where operator's policy splits could produce 2+.
- Each invoice's metadata JSONB carries originating_workflow field (e.g., SUB-WF-RELOCATION-01:wf_01HZQA1...) for traceability back to the workflow that issued the fee.
- InvoiceIssued event with the same self-contained payload (customer_snapshot + line items + tax).

Failure paths: same as cycle generator. PREPAID one-off invoice failures are particularly important to monitor because the customer is expected to pay this explicitly — if the invoice doesn't generate, the customer doesn't know they owe the fee.

Generator 3 — ProFormaCycleGenerator

Purpose: produce pro forma documents for PREPAID subscriptions ~5 days before their cycle close, so customers know how much will be settled at cycle end.

Trigger: Scheduled scanner at 03:00 local-time per operator (NOT Kafka-driven — pro forma is system-initiated, not event-initiated).

Scanner mechanics:

```
-- Conceptual query (run per operator at 03:00 local time)
SELECT s.id, s.customer_id, s.operator_code, s.current_cycle_start, s.current_cycle_end
FROM subscription_master s
WHERE s.operator_code = :operator
  AND s.billing_mode = 'PREPAID'
  AND s.status NOT IN ('TERMINATED', 'SUSPENDED', 'PENDING_NEW',
                      'PENDING_UPGRADE', 'PENDING_DOWNGRADE', 'PENDING_TERMINATION')
  AND (s.current_cycle_end - NOW()) <= (
    SELECT pre_cycle_pro_forma_window_days * INTERVAL '1 day'
    FROM invoice_type_operator_config
    WHERE operator_code = :operator
      AND invoice_type_code = 'PRO_FORMA_CYCLE'
  )
  AND NOT EXISTS (
    SELECT 1 FROM pro_forma pf
    WHERE pf.subscription_id = s.id
      AND pf.idempotency_cycle_key = 'cycle_' || EXTRACT(YEAR FROM s.current_cycle_end + INTERVAL '1 day') || '_'
      || LPAD(EXTRACT(MONTH FROM s.current_cycle_end + INTERVAL '1 day')::TEXT, 2, '0') ||
      || 'sub_' || s.id
  );
```

For each row returned, the scanner builds an `InvoiceGenerationContext` and calls `InvoiceGenerationService.generate(context)`.

Specific rules:

ID	Rule	Triggered on
R-GEN-01-P-1	Generator runs as scheduled scanner at 03:00 local-time per operator. Scans for PREPAID subscriptions in valid state with cycle end within <code>pre_cycle_pro_forma_window_days</code> AND no pro forma already generated for the upcoming cycle.	Scanner trigger
R-GEN-01-P-2	Generator calls BIL-01 Charging Engine with intent: <code>PRO_FORMA_PREVIEW</code> and the upcoming cycle window. BIL-01 returns projected charges (best estimate — usage components based on current cycle's run-rate, fixed components exact).	Charge projection
R-GEN-01-P-3	Pro forma generated per operator's grouping policy for <code>PRO_FORMA_CYCLE</code> . Same dimension rules as cycle invoice — hybrid policy produces multiple pro forma documents.	Grouping policy
R-GEN-01-P-4	<code>idempotency_cycle_key = cycle_{YYYY}_{MM}_sub_{subscription_id}</code> from the upcoming cycle window. Combined with <code>grouping_config_ref</code> , used by the scanner to skip subscriptions whose pro forma is already generated.	Idempotency key
R-GEN-01-P-5	After successful generation, generator writes <code>current_cycle_pro_forma_id</code> back to Subscription master via SUB-LM-01 <code>put-master-fields</code> .	Master field writeback
R-GEN-01-P-6	Customer snapshot captured fresh at pro forma generation time. Different from the eventual cycle invoice's snapshot (taken ~5 days later).	Snapshot timing
R-GEN-01-P-7	When next pro forma is generated for the same subscription, previous pro forma's status → <code>SUPERSEDED</code> , <code>superseded_by_id</code> populated. Transition part of new pro forma's write transaction.	Pro forma supersession

End-to-end flow (one scanner tick → 0..N pro forma per subscription):

```

Step 1 – Scanner wakes up at 03:00 operator-local
  For each operator, run the conceptual query above; receive list of subscriptions due for pro forma.

Step 2 – Per subscription, generate pro forma
  For each subscriptionId returned:
    Step 2a – Build InvoiceGenerationContext
      Set invoiceTypeCode = PRO_FORMA_CYCLE, subscriptionId, operatorCode,
      cycleWindow = next cycle (cycle_start = current cycle_end + 1ms, cycle_end = next month boundary),
      triggeringEventRef = `scanner_tick_{operator}_{date}` (synthetic, for traceability).

    Step 2b – Subscription state gate (R-GEN-01-F-4)
      Re-check state (it may have changed between scanner query and now).
      PENDING → enqueue gen_failure_queue, continue to next subscription.

    Step 2c – Compute idempotency_cycle_key
      Construct as: `cycle_{nextCycleYear}_{nextCycleMonth}_sub_{subscriptionId}`.

    Step 2d – Capture customer snapshot (R-GEN-01-F-6)
      Failure → enqueue, continue.

    Step 2e – Fetch projected charges from BIL-01 (R-GEN-01-P-2)
      POST /api/billing/charges/compute with { subscriptionId, intent: PRO_FORMA_PREVIEW, cyclewindow }.
      Receive projected charges (fixed + estimated usage).
      Empty → no pro forma for this customer (their next cycle would have no charges).
      Failure → enqueue, continue.

    Step 2f – Load grouping policy for (operatorCode, PRO_FORMA_CYCLE) (R-GEN-01-P-3)
      Receive 1..N groups.

    Step 2g – Per group, generate one pro forma
      For each group:
        - Idempotency check on (subscriptionId, idempotency_cycle_key, group.id) → return existing if found
        - Filter charges by group's keys
        - Build line items via InvoiceLineBuilder (same as invoice – SUMMARY + DETAIL + tax)
        - Build pro_forma header (with billing_mode = PREPAID, idempotency_cycle_key, etc.)
        - Allocate pro_forma sequence (non-gap-free; pro forma doesn't require gap-free)

```

- Write atomically (sequence + pro_forma row + line items + ProFormaIssued outbox event)

Step 2h – After all groups for this subscription:

Supersede previous pro forma (if any) → status SUPERSEDED, superseded_by_id = new pro forma id.
Write current_cycle_pro_forma_id back to Subscription master via SUB-LM-01 put-master-fields.

Step 3 – Scanner completes

Log summary: subscriptions scanned, pro forma generated, failures.
Wait until 03:00 next day (operator-local).

Output:

- 0..N rows in pro_forma table per subscription per scanner run
- 0..N ProFormaIssued events (consumed by DD_NOT-01 Notification Service for SMS/email "your wallet has X, top up Y before cycle Z")
- Previous pro forma rows for the same subscription transitioned to SUPERSEDED
- subscription_master.current_cycle_pro_forma_id updated

Failure paths:

Where it fails	Lands in	Recovery
Subscription PENDING at scan time	generation_failure_queue	Next scanner run picks it up if still due
BIL-01 unavailable	generation_failure_queue	Retry scanner
CRM unavailable	generation_failure_queue	Retry scanner
Tax service down	generation_failure_queue	Retry scanner
DB write fails	generation_failure_queue (no sequence gap audit since pro forma sequence is non-gap-free)	Retry scanner

Note: Pro forma is informational — failed pro forma generation does NOT prevent cycle close. If a pro forma for cycle N+1 fails to generate, the customer just doesn't get a heads-up SMS; cycle N+1's actual settlement still happens at cycle close via BIL-05 Wallet and Topup. Operator alerting fires if pro forma failure rate exceeds 1% per operator-day (configurable).

Generator 4 — CreditNoteGenerator

Purpose: produce a credit note that reduces a customer's outstanding balance on a previously-issued invoice. Triggered by BIL-02-ADJ-01 Invoice Adjustments after an adjustment is captured and approved.

Trigger: In-process Java method call from BIL-02-ADJ-01 Invoice Adjustments. NOT Kafka-driven. ADJ-01 builds an InvoiceGenerationContext and calls InvoiceGenerationService.generate(context) synchronously, expecting the resulting invoice in the return value.

Context payload from ADJ-01:

```
InvoiceGenerationContext ctx = InvoiceGenerationContext.builder()
    .invoiceTypeCode("CREDIT_NOTE")
    .triggeringEventRef("adjustment_" + adjustmentId) // synthetic, derived from adjustment ID
    .subscriptionId(originalInvoice.getSubscriptionId())
    .operatorCode(originalInvoice.getOperatorCode())
    .originalInvoiceId(originalInvoice.getId())
    .adjustmentRequest(adjustmentRequest) // contains: affected line items, credit amounts per line, reason
    .adminContext(approvalContext) // who approved, when
    .build();
```

Specific rules (shared with debit note in group N):

ID	Rule	Triggered on
R-GEN-01-N-1	Generator invoked synchronously by BIL-02-ADJ-01 Invoice Adjustments via Java method call (not Kafka). ADJ-01 builds the context with adjustment details.	Trigger source
R-GEN-01-N-2	Credit/debit note carries original_invoice_id pointing to the invoice being adjusted (mandatory FK). Customer snapshot captured fresh (may differ from original invoice's snapshot). grouping_config_ref and grouping_dimension copied from original — note reflects same grouping.	Linkage to original

ID	Rule	Triggered on
R-GEN-01-N-3	Line items computed from adjustment context: which original line items are affected and what credit/debit amounts apply. Builder produces SUMMARY/DETAIL with negative amounts for credit / positive for debit. PLM-CFG-02 TaxService recomputes tax on adjusted amounts (NOT linearly scaled — tax rules may have non-linear effects like exemption thresholds).	Line construction
R-GEN-01-N-4	Own legal invoice number sequence (CREDIT_NOTE / DEBIT_NOTE types). Format operator-configured (e.g., WIK-CN-2026-NNNNNNNN for credit notes Wananchi KE). Gap-free same as other legal documents.	Numbering
R-GEN-01-N-5	billing_mode copied from original invoice. triggering_event_ref = adjustment_{adjustment_id} for idempotency.	Header fields
R-GEN-01-N-6	Resulting invoice ID returned to ADJ-01; ADJ-01 stores it in invoice_adjustment.adjustment_invoice_id. InvoiceIssued event emitted with metadata.adjustment_id populated for downstream correlation.	Return + event

End-to-end flow (one adjustment approval → one credit note):

Step 1 – ADJ-01 calls InvoiceGenerationService.generate(context)
After approval workflow completes, ADJ-01 calls synchronously from its own transaction context.

Step 2 – Dispatch to CreditNoteGenerator
Registry finds the bean by typeCode = CREDIT_NOTE.

Step 3 – Idempotency check (R-GEN-01-F-5)
Look up existing invoice with (triggering_event_ref = adjustment_X, CREDIT_NOTE, group.id from original).
Found → return existing. (Very rare: re-running the same adjustment approval.)

Step 4 – Load original invoice
Fetch the row by original_invoice_id; will need its grouping_config_ref, billing_mode, currency, etc.

Step 5 – Capture customer snapshot (R-GEN-01-F-6)
Fresh snapshot at credit note issuance moment. NOT copied from original invoice (R-GEN-01-N-2).
Failure → throw exception back to ADJ-01 (synchronous call; ADJ-01 decides retry vs queue).

Step 6 – Build line items (R-GEN-01-N-3 + R-GEN-01-L-*)
For each adjustmentLine in adjustmentRequest:
- Match against original invoice's line item (by original_line_id reference)
- Compute credit amount (negative subtotal)
- Call PLM-CFG-02 TaxService.computeTax with the negative amount → returns negative tax
- Build a single DETAIL line referencing the original via metadata.original_line_id
Build a SUMMARY line per affected wallet/package per the original's grouping dimension.
Aggregate tax_summary from per-line negative tax breakdowns.

Step 7 – Build credit note header
invoice.original_invoice_id = original.id
invoice.subscriptionId, customerId, operatorCode, currency: from original
invoice.billing_mode: copied from original
invoice.grouping_config_ref + grouping_dimension + grouping_key_values: copied from original
invoice.customer_snapshot: from step 5 (fresh)
invoice.triggering_event_ref: `adjustment\_{adjustmentId}`
invoice.issue_date: now()
invoice.due_date: NULL (credit note is not "due" – it's a balance reduction)
invoice.subtotal_amount, tax_amount_total, total_amount: negative
invoice.status: GENERATED
invoice.metadata: { adjustment_id, original_invoice_number, approved_by, approval_timestamp }

Step 8 – Allocate CREDIT_NOTE sequence + write atomically (R-GEN-01-F-7)
Sequence service allocates from CREDIT_NOTE sequence.
Single DB transaction:
- sequence row update
- invoice row insert (negative amounts)
- invoice_line_item bulk insert
- outbox event row (InvoiceIssued with metadata.adjustment_id)
Failure → invoice_sequence_gap row + throw exception back to ADJ-01.

Step 9 – Return the persisted credit note invoice to ADJ-01
ADJ-01 stores credit_note.id in invoice_adjustment.adjustment_invoice_id within its own transaction.

Output:

- 1 row in `invoice` table with negative amounts and `original_invoice_id` set
- N rows in `invoice_line_item` with negative line subtotals and negative tax
- 1 `InvoiceIssued` event with `metadata.adjustment_id` populated
- BIL-01 Charging Engine consumes the event and reduces outstanding balance against the original invoice
- BIL-04 Dunning Engine consumes the event and may stop dunning if the credit note fully clears the original's balance
- DD_NOT-01 Notification Service consumes the event and sends the credit note to the customer

Failure paths: identical to cycle generator's, except failures are thrown back to ADJ-01 synchronously (since this is a synchronous Java call, not a Kafka consumer path). ADJ-01 decides what to do — retry inline, or queue its own adjustment for re-processing. ADJ-01 owns that decision.

Generator 5 — DebitNoteGenerator

Purpose: produce a debit note that increases a customer's outstanding balance on a previously-issued invoice. Used for under-charges discovered after the fact (missed line items, tax under-applied, regulatory adjustments increasing tax retroactively, etc.).

Trigger: In-process Java method call from BIL-02-ADJ-01 Invoice Adjustments. Same mechanism as credit note but with positive amounts.

Specific rules: same group N rules as credit note (shared between the two generators).

End-to-end flow: identical to credit note's, with these differences in step 6:

- `adjustmentLine.amount` is positive (debit means "customer owes more")
- `TaxService` returns positive tax on the positive base
- Resulting line subtotals and tax amounts are positive
- Aggregated invoice total is positive

In step 7, the debit note header has:

- `invoice.due_date = now() + grace_days` (debit note IS due — customer owes more, has time to pay)
- `total_amount` positive
- `metadata`: same structure as credit note but `adjustment_type = DEBIT`

Output: 1 row in `invoice` with positive amounts, `original_invoice_id` set, due date set. BIL-01 increases outstanding balance. BIL-04 may start dunning if the debit note isn't paid by due date.

Failure paths: identical to credit note's.

3. Module owner and impacted modules

Role	Module	What it does
Owner	New billing-invoicing-gen package within the billing-invoicing module on cluster C	Co-located with BIL-02 Core. Owns the 5 generator classes, the line item builder, the scheduled pro forma scanner, the bulk reversal service, and the generation failure queue scanner.
Upstream events consumed	BIL-03 Cycle Close (<code>CycleClosed</code>), subscription workflows + admin (<code>OneOffChargeRequested</code>)	Trigger sources for cycle and one-off invoices.
Synchronous callees	BIL-01 Charging Engine (charge computation), PLM-CFG-02 Tax Configuration (tax computation), SUB-LM-01 Subscription Lifecycle Management (state checks + master field writeback), CRM/customer service (customer snapshot), BIL-02 Core (sequence service, invoice writer, pro forma writer, event publisher)	All synchronous REST or in-process calls.
Downstream events emitted	<code>InvoiceIssued</code> , <code>ProFormaIssued</code> (via Core's event publisher), <code>BulkReversalCompleted</code> (GEN-01's own event), <code>GenerationFailureGivenUp</code> (operator alert)	All events go out via the Kafka outbox in the same transaction as the data write.

Role	Module	What it does
Frontend	Admin Console — bulk reversal initiation + preview + monitoring; generation failure queue review + manual retry/resolve; manual one-off invoice issuance	Built on top of BIL-02 Core's admin shell.
Engines	No Drools in GEN-01. Tax rules in PLM-CFG-02 Tax Configuration (called via REST).	—

4. Foundation references

DB → FOUNDATION_DB.md · Kafka → FOUNDATION_KAFKA.md · Auth → FOUNDATION_AUTH.md. Framework: SUB-WF-FRAMEWORK-01 Subscription Workflow Framework + BIL-02 Invoicing (Core).

Technical design

Module placement

Co-located in billing-invoicing module on cluster C. New sub-package gen containing generators, builder, scanners, and the bulk reversal service. Shares BIL-02 Core's data sources (no new DB instances); adds 3 new tables (bulk reversal batch + failure + generation failure queue).

Code structure

```
modules/billing-invoicing/ (shared with Core + satellites)
  src/main/java/com/sophix/billing/invoicing/gen/
    service/
      CyclePostpaidInvoiceGenerator.java
      OneOffInvoiceGenerator.java
      ProFormaCycleGenerator.java
      CreditNoteGenerator.java
      DebitNoteGenerator.java
      InvoiceLineBuilder.java
      BulkReversalService.java
    scanner/
      ProFormaScheduledScanner.java      ← daily at 03:00 per operator
      GenerationFailureRetryScanner.java ← every 15 minutes
    kafka/
      CycleClosedConsumer.java           ← filters POSTPAID
      OneOffChargeRequestedConsumer.java ← filters payment_handling = ISSUE_INVOICE
    rest/
      BulkReversalResource.java           ← admin endpoints
      GenerationFailureQueueResource.java ← admin endpoints
      ManualOneOffInvoiceResource.java    ← admin endpoint for manual issuance
    domain/
      BulkReversalBatch.java
      BulkReversalFailure.java
      GenerationFailureEntry.java
    repository/
      BulkReversalBatchRepository.java
      GenerationFailureQueueRepository.java
  src/main/resources/
    config/liquibase/changelog/billing-invoicing/
      040_bulk_reversal_batch.xml
      041_bulk_reversal_failure.xml
      042_generation_failure_queue.xml
```

Additional data model (beyond Core)

GEN-01 introduces three new tables in addition to what Core provides.

bulk_reversal_batch

Tracks an operator-initiated bulk reversal operation.

Column	Type	Notes
id	TEXT PK	ULID
operator_code	TEXT NOT NULL	Which operator's invoices are being reversed
scope_invoice_type_code	TEXT NOT NULL	Which type was reversed (CYCLE_POSTPAID, ONE_OFF_INVOICE, etc.)
scope_date_from	DATE NOT NULL	Lower bound on issue_date

Column	Type	Notes
scope_date_to	DATE NOT NULL	Upper bound on issue_date
scope_filter	JSONB NULL	Optional additional filters: package_ref list, wallet_type_code list, service_category_code list, customer_id list
re_issue	BOOLEAN NOT NULL	If true, re-invoke generators with corrected inputs after cancellation
requested_by	TEXT NOT NULL	Billing Admin user ID
requested_at	TIMESTAMPTZ NOT NULL	
approved_by	TEXT NOT NULL	Finance Head user ID
approved_at	TIMESTAMPTZ NOT NULL	
started_at	TIMESTAMPTZ NULL	When execution started after both approvals
completed_at	TIMESTAMPTZ NULL	When all in-scope invoices were processed
invoices_in_scope	INTEGER	Counted at preview
invoices_cancelled	INTEGER	
invoices_re_issued	INTEGER	
invoices_failed	INTEGER	
status	TEXT NOT NULL	PREVIEW / PENDING_APPROVAL / IN_PROGRESS / COMPLETED / PARTIALLY_FAILED / CANCELLED
failure_summary	JSONB NULL	High-level summary of failures (top reason codes + counts)
notes	TEXT NULL	Operator notes explaining why the batch was issued

Indexes: (operator_code, status, requested_at DESC) for admin queue. P10Y retention. Change history table bulk_reversal_batch_history tracks status changes.

Sample data:

id	operator_code	scope_invoice_type_code	scope_date_from	scope_date_to	re_issue	requested_by	approved_by	invoices_in_scope	invoices_cancelled	invoices_re_issued	status
brb_01HZ QR...	WANANCHI_KE	CYCLE_POSTPAID	2026-09-01	2026-09-30	true	admin_billing_grace_m	finance_head_john_k	12,847	12,840	12,840	COMPLETED
brb_01HZ QS...	WANANCHI_KE	ONE_OFF_INVOICE	2026-10-01	2026-10-15	false	admin_billing_peter_w	finance_head_john_k	342	340	0	PARTIALLY_FAILED

How to read: the first batch reversed all September POSTPAID cycle invoices for Wananchi KE — 12,840 of 12,847 succeeded, 7 failed (those failures are detailed in bulk_reversal_failure). The reason a Billing Admin would do this: a tax rule was misconfigured for September, so all the cycle invoices computed wrong tax. Re-issue is true, so after cancellation the corrected invoices were generated. The second batch reversed October one-off invoices (perhaps a wrong relocation fee schedule was loaded) but with re_issue = false — the operator decided to manually re-issue them via a different process.

bulk_reversal_failure

Per-invoice failure detail for a bulk reversal batch. Populated for invoices that failed cancellation or re-issuance.

Column	Type	Notes
id	TEXT PK	ULID
batch_id	TEXT NOT NULL	FK to bulk_reversal_batch.id
invoice_id	TEXT NOT NULL	The invoice that failed
phase	TEXT NOT NULL	CANCEL (cancellation step failed) or RE_ISSUE (cancellation succeeded but re-generation failed)
reason_code	TEXT NOT NULL	e.g., INVOICE_IN_PROTECTED_STATE, BIL01_UNAVAILABLE, TAX_SERVICE_FAILED, CUSTOMER_SNAPSHOT_FETCH_FAILED
reason_detail	TEXT NULL	Free-form (exception summary)
failed_at	TIMESTAMPTZ NOT NULL	

Column	Type	Notes
admin_resolution	TEXT NULL	Notes from admin who reviewed (e.g., "manually cancelled via admin console on 2026-10-12, ref CHK-2026-Q4-FIX-001")
resolved_at	TIMESTAMPTZ NULL	
resolved_by	TEXT NULL	

Indexes: (batch_id); (reason_code, failed_at) for analytics. P10Y retention.

Sample data (for batch brb_01HZQS... showing 2 of the 342 failures):

id	batch_id	invoice_id	phase	reason_code	reason_detail	failed_at	admin_resolution
brf_01...A	brb_01HZQS...	inv_01HVN...	CANCEL	INVOICE_IN_PROTECTED_STATE	Linked to credit note CN-2026-00478; cannot bulk-reverse	2026-10-15 14:22:01	(NULL — pending review)
brf_01...B	brb_01HZQS...	inv_01HVNY...	CANCEL	INVOICE_IN_PROTECTED_STATE	Status already CANCELLED	2026-10-15 14:22:01	(NULL — pending review)

How to read: bulk reversals skip invoices in protected states. The admin reviews these in the failure queue and decides individually — maybe cancel the linked credit note first then come back, or accept that the invoice can't be reversed.

generation_failure_queue

Captures recoverable generation failures for automated retry and admin review.

Column	Type	Notes
id	TEXT PK	ULID
operator_code	TEXT NOT NULL	
invoice_type_code	TEXT NOT NULL	What was being generated (CYCLE_POSTPAID, ONE_OFF_INVOICE, PRO_FORMA_CYCLE, CREDIT_NOTE, DEBIT_NOTE)
triggering_event_ref	TEXT NOT NULL	The original trigger (Kafka event ID, scanner tick ID, or adjustment_{id})
subscription_id	TEXT NOT NULL	Subscription the generation was for
trigger_payload	JSONB NOT NULL	Full original trigger context (so retries don't depend on re-fetching the original event from Kafka)
reason_code	TEXT NOT NULL	SUBSCRIPTION_PENDING / BIL01_UNAVAILABLE / TAX_SERVICE_FAILED / CUSTOMER_SNAPSHOT_FETCH_FAILED / SEQUENCE_ALLOCATION_FAILED / DB_TRANSIENT_ERROR (extensible)
reason_detail	TEXT NULL	Free-form (exception summary or message)
retry_count	INTEGER NOT NULL DEFAULT 0	Number of retries so far
max_retries	INTEGER NOT NULL DEFAULT 8	Operator-configurable per failure type if needed
next_retry_at	TIMESTAMPTZ NULL	When the scanner will pick this up; NULL when in terminal state
created_at	TIMESTAMPTZ NOT NULL	When first enqueued
last_attempt_at	TIMESTAMPTZ NULL	When the most recent attempt happened
status	TEXT NOT NULL DEFAULT 'PENDING_RETRY'	PENDING_RETRY / RETRIED_SUCCESS / GAVE_UP_AUTO / RESOLVED_NO_ACTION / RESOLVED_MANUAL_RETRY_QUEUED
resolution_notes	TEXT NULL	Admin notes if status is RESOLVED_NO_ACTION
resolved_by	TEXT NULL	
resolved_at	TIMESTAMPTZ NULL	

Indexes: (status, next_retry_at) partial WHERE status='PENDING_RETRY' for the retry scanner; (operator_code, status, created_at DESC) for admin queue. P10Y retention.

Sample data:

id	invoice_type_code	subscription_id	reason_code	retry_count	next_retry_at	status
gfq_01...A	CYCLE_POSTPAID	sub_01HVR3T2W5K8	SUBSCRIPTION_PENDING	2	2026-10-01 04:01:00	PENDING_RETRY
gfq_01...B	PRO_FORMA_CYCLE	sub_01HVR8M4N2P6	BIL01_UNAVAILABLE	1	2026-09-26 03:45:00	PENDING_RETRY
gfq_01...C	ONE_OFF_INVOICE	sub_01HVR9P5Q3R2	CUSTOMER_SNAPSHOT_FETCH_FAILED	8	NULL	GAVE_UP_AUTO

How to read: the first entry is a cycle invoice that couldn't generate because the subscription was in `PENDING_UPGRADE` state — the scanner will keep retrying every 15-30 minutes until the workflow completes. The second is a transient BIL-01 outage — already retried once, scheduled again shortly. The third hit the retry cap (8 attempts over ~3 days) without resolution — moved to `GAVE_UP_AUTO`, awaits admin review (perhaps the customer record was deleted in CRM, which is why the snapshot fetch keeps failing).

API surface — what GEN-01 exposes

GEN-01's external surface is **mostly event-driven**, not REST-driven. This is deliberate: invoice generation is triggered by business events (cycle close happened, a workflow needs a fee, a scheduler woke up, an adjustment was approved), not by HTTP requests. The REST endpoints that do exist are operational (admin-facing) — they don't drive normal day-to-day generation.

Three integration mechanisms with clear boundaries:

Mechanism	Used for	Examples
Kafka consumption (events in)	Normal generation triggered by business events	CycleClosed → cycle invoice; OneOffChargeRequested → one-off invoice
Scheduled scanner (no inbound API)	Pro forma generation (system-initiated, not event-driven)	Daily 03:00 operator-local scanner producing pro forma
In-process Java method call	Synchronous callout from a sibling DD	BIL-02-ADJ-01 calls <code>InvoiceGenerationService.generate(creditNoteContext)</code> and gets the resulting invoice back
Kafka emission (events out)	Notifying downstream consumers	InvoiceIssued, ProFormaIssued, BulkReversalCompleted, GenerationFailureGivenUp
REST (admin-facing)	Operational endpoints used by ops/billing admin staff via admin console	Bulk reversal management, generation failure queue review, manual one-off invoice issuance

What GEN-01 does NOT expose as a REST endpoint (by design):

Capability	Why not here	Where it lives instead
GET <code>/api/invoices/{id}</code> (read an invoice)	Read APIs for invoice retrieval, search, listing belong to the satellite that owns invoice access	BIL-02-READ-01 Invoice Read API owns invoice GET/list APIs
GET <code>/api/invoices/{id}/status</code> (check status)	Same — reading invoice state, listing by status, etc.	BIL-02-READ-01 Invoice Read API
GET <code>/api/invoices/{id}/pdf</code> (download PDF)	Same	BIL-02-READ-01 Invoice Read API
POST <code>/api/invoices/{id}/cancel</code> (cancel an invoice)	Invoice status transitions after generation belong to lifecycle management	BIL-02-STA-01 Status Management & Recovery
POST <code>/api/invoices/cycle/generate</code> (force cycle generation for a specific subscription)	Cycle generation is event-driven from BIL-03 Cycle Close; admin can replay the cycle close event if they need to force regeneration	Use generation failure queue manual retry or replay the Kafka event upstream
POST <code>/api/invoices/oneoff</code> (admin one-off)	This DOES exist as POST <code>/api/invoices/manual</code> — see REST section	(kept in GEN-01)
POST <code>/api/credit-notes</code> (create credit note)	Adjustments have their own workflow with approval	BIL-02-ADJ-01 Invoice Adjustments owns the admin flow that ultimately calls GEN-01's <code>CreditNoteGenerator</code>

So the **only** GEN-01 REST endpoints that exist are the operational/admin ones listed in the next subsection. Day-to-day generation needs no REST surface because it's all event-driven.

GEN-01 mostly emits events through BIL-02 Core's event publisher (`InvoiceIssued`, `ProFormaIssued`). GEN-01-specific events are limited to bulk reversal and generation failures.

Topic	When	Consumers
sophix.billing.invoice.bulkreversal.completed	A bulk_reversal_batch finishes (status moves to COMPLETED or PARTIALLY_FAILED)	analytics, audit, operator alerting
sophix.billing.invoice.generation.giveup	A generation_failure_queue entry moves to GAVE_UP_AUTO	operator alerting (admin needs to review)

GEN-01 consumes the following events from upstream:

Topic	From	Action
sophix.billing.cycle.closed	BIL-03 Cycle Close	Trigger CyclePostpaidInvoiceGenerator (filtered for POSTPAID)
sophix.billing.charge.oneoff.requested	subscription workflows + admin	Trigger OneOffInvoiceGenerator (filtered for payment_handling = ISSUE_INVOICE)

REST endpoints (admin-facing — operational only)

These are the **only** REST endpoints GEN-01 exposes. They support admin workflows (bulk reversal, failure queue review, manual one-off issuance). All other invoice operations go through the appropriate sibling DD as listed in the API surface table above.

GET /api/bulk-reversals/preview?...	← preview scope match count + total amount
POST /api/bulk-reversals	← initiate (Billing Admin); status = PENDING_APPROVAL
POST /api/bulk-reversals/{id}/approve	← approve (Finance Head); transitions to IN_PROGRESS
POST /api/bulk-reversals/{id}/cancel	← cancel before execution starts
GET /api/bulk-reversals/{id}	← batch detail + counts
GET /api/bulk-reversals/{id}/failures	← list of per-invoice failures
GET /api/bulk-reversals?operator={code}&status={s}	← admin listing
GET /api/generation-failures	← queue listing
GET /api/generation-failures/{id}	← entry detail
POST /api/generation-failures/{id}/retry	← manual retry trigger
POST /api/generation-failures/{id}/resolve-no-action	← admin decides not to generate
POST /api/invoices/manual	← admin manually issues a one-off invoice (alternative path)

Cross-DD coordination

Touched	What GEN-01 owns	What the other DD owns
BIL-02 Invoicing (Core)	Implements generator classes per the framework; uses sequence service, invoice/pro forma writer, customer snapshot service, line item infrastructure, event publisher.	Owns the framework + entity + lifecycle + catalog + sequence allocator.
BIL-02-TAX-01 Tax Invoice & Gateway	Does not invoke or interact with TAX-01 directly. Tax invoice generation is triggered by its own consumer (PaymentApplied), not by GEN-01's events.	Owns tax invoice generator (separate generator class), signing flow, retry.
BIL-02-ADJ-01 Invoice Adjustments	Provides CreditNoteGenerator and DebitNoteGenerator that ADJ-01 calls in-process after approving an adjustment.	Owns adjustment capture, approval workflow, threshold routing.
BIL-02-READ-01 Invoice Read API	None directly. After invoice generation, READ-01 consumes the InvoiceIssued event for cache invalidation, and serves the resulting invoice through its unified read API. PDF rendering itself is owned by DD_NOT-01 Notification Service.	Owns invoice read API + PDF stream serving.
BIL-02-STA-01 Status Management & Recovery	None directly during generation. After generation, STA-01 owns lifecycle transitions (PENDING → PAID/OVERDUE, dunning, etc.).	Owns lifecycle state machine after invoice exists.
BIL-01 Charging Engine	Calls POST /api/billing/charges/compute synchronously for cycle and one-off intents; calls with intent: PRO_FORMA_PREVIEW for pro forma.	Owns charging logic + amount computation + outstanding balance.
BIL-03 Cycle Close	Consumes CycleClosed events for POSTPAID cycle invoice generation.	Owns cycle window detection + emission.
BIL-04 Dunning Engine	None directly. BIL-04 consumes invoice events emitted via BIL-02 Core.	Owns dunning state machine.

Touched	What GEN-01 owns	What the other DD owns
BIL-05 Wallet and Topup	None directly in the cycle invoice path (POSTPAID-only). For PREPAID one-off charges with <code>payment_handling = WALLET_DEBIT</code> , GEN-01 does NOT consume — BIL-05 consumes directly.	Owns wallet entity + cycle settlement for PREPAID.
SUB-LM-01 Subscription Lifecycle Management	Calls for subscription status snapshot at generation time; writes <code>current_cycle_pro_forma_id</code> back via <code>put-master-fields</code> after pro forma generation.	Owns subscription master fields.
Subscription workflows (SUB-WF-RELOCATION-01, SUB-WF-UPGRADE-01, SUB-WF-EQUIPMENT-SWAP-01, etc.)	Consumes their <code>OneOffChargeRequested</code> events.	Emit the events when their workflow incurs a fee.
PLM-CFG-01 Package Catalog	Reads package metadata when applying PACKAGE-dimension grouping policy and for translation keys.	Owns package definitions.
PLM-CFG-02 Tax Configuration	Calls <code>TaxService.computeTax</code> per line item at generation time.	Owns tax rules.
PLM-CFG-03 Wallet Configuration	Reads wallet type catalog when applying WALLET-dimension grouping policy.	Owns wallet types.
PLM-CFG-04 Service Catalog	Reads service category catalog when applying SERVICE_CATEGORY-dimension grouping policy.	Owns service categories.
CRM / Customer Service	Called via BIL-02 Core's <code>CustomerSnapshotService</code> for customer details at generation time.	Owns live customer master.
DD_NOT-01 Notification Service	None directly — Core's event publisher emits the <code>InvoiceIssued</code> / <code>ProFormaIssued</code> events that DD_NOT-01 consumes for rendering and dispatch.	Owns template rendering + channel dispatch.

Dependencies

Dependency	Owner
BIL-02 Invoicing (Core)	billing team (same wave)
BIL-01 Charging Engine	billing team
BIL-03 Cycle Close	billing team
BIL-05 Wallet and Topup	billing team
SUB-WF-FRAMEWORK-01 Subscription Workflow Framework	subscription team
SUB-LM-01 Subscription Lifecycle Management	subscription team
Subscription workflows emitting <code>OneOffChargeRequested</code>	subscription team
PLM-CFG-01 Package Catalog	catalog team
PLM-CFG-02 Tax Configuration	catalog team
PLM-CFG-03 Wallet Configuration	catalog team
PLM-CFG-04 Service Catalog	catalog team
CRM / Customer Service module	CRM team
FOUNDATION_DB.md, FOUNDATION_KAFKA.md, FOUNDATION_AUTH.md	infrastructure team
BIL-02-ADJ-01 Invoice Adjustments (calls into GEN-01)	billing team (same wave)
DD_NOT-01 Notification Service	notification team

Operator-configurable capabilities (specific to GEN-01)

Beyond what BIL-02 Core exposes, GEN-01 has these per-operator knobs:

Capability	Where configured	Notes
Pro forma scanner schedule	Per operator local time	Defaults to 03:00 operator-local; configurable
Pre-cycle pro forma window	<code>invoice_type_operator_config.pre_cycle_pro_forma_window_days</code> (in Core)	Used by GEN-01's pro forma scanner
Max retries in generation failure queue	per <code>reason_code</code> , in operator config	Different reasons can have different retry limits

Capability	Where configured	Notes
Bulk reversal authorization roles	role mapping in FOUNDATION_AUTH.md	Defaults: Billing Admin + Finance Head; operators can require additional approvers
One-off charge handling per fee type	Per fee type, configured by the workflow module that emits the charge	The fee descriptor on the OneOffChargeRequested event carries the payment_handling choice

B — Current TZ

In TZ Sophix, invoice generation is embedded in the billing-engine cron job — a single monolithic procedure that on a schedule queries due cycles, computes charges, applies tax, and writes invoices. There is no formal separation of generators per invoice type; everything goes through the same code path. Pro forma is implemented as ad-hoc database queries rendered by an analytics tool rather than as a first-class document with sequence numbering and lifecycle. One-off invoices for relocation and equipment swap exist but are issued by direct database inserts from the workflow handlers, bypassing the cron — so they have inconsistent sequence numbering, inconsistent tax treatment, and no idempotency.

Bulk reversal exists as a manual stored-procedure that an operator runs from a database client; there is no preview, no approval flow, no per-invoice audit, and re-issuance is a separate manual step. The reversal procedure is invoked perhaps 2-3 times per year, always followed by a multi-hour cleanup of inconsistencies.

There is no generation failure queue. When the cron job fails partway through a cycle close — usually because the tax microservice is unresponsive or BIL-01's predecessor times out — the operations team manually identifies missing invoices via spreadsheet comparison against the subscription list and triggers a partial re-run of the cron. This process loses 1-2 invoices per quarter to operational errors.

Idempotency is partial. The cron uses a "last processed cycle timestamp" to avoid re-processing the same cycle, but mid-cycle Kafka retries (the cron does consume some Kafka events) sometimes produce duplicate invoices that have to be manually cancelled.

C — Gap & bridge

Changes from TZ → V3

Topic	TZ Sophix	V3 (GEN-01)
Generator separation	Monolithic cron procedure	One class per invoice type, dispatched via the InvoiceGenerator framework
Pro forma	Ad-hoc analytics query	First-class scheduled scanner producing real pro_forma rows with sequence, lifecycle, supersession
One-off invoice	Workflow-driven direct DB inserts	Kafka-event-driven generator with idempotency and per-fee-type payment handling
Bulk reversal	Manual stored procedure, no audit	First-class capability with preview, dual approval, per-invoice transactional execution, full audit (bulk_reversal_batch + bulk_reversal_failure)
Generation failure handling	Manual spreadsheet reconciliation	generation_failure_queue with automatic retry scanner + exponential backoff + admin review queue
Idempotency	Partial (cycle timestamp only)	Universal across all generators via (triggering_event_ref, invoice_type_code, grouping_config_ref) for invoices and (subscription_id, idempotency_cycle_key, grouping_config_ref) for pro forma
Grouping policy support	Hardcoded (Wananchi-specific Internet+TV / Voice split coded in the cron)	Dimension-driven policy applied by every generator
Customer snapshot	None (re-fetches from CRM at PDF render time)	Captured immutably at generation per Core R-C-1
Tax computation	Inline	Delegated to PLM-CFG-02 Tax Configuration per line via the line builder
Translation	Render-time	At generation time, translation keys resolved against operator catalog and stored on the line

What's NEW in V3 (not in TZ)

1. **Per-type generator classes** with the InvoiceGenerator framework, registered and dispatched by Core
2. **Pro forma as a first-class scanned document** with sequence numbering, lifecycle, supersession
3. **One-off invoice generator** triggered by workflow events with per-fee-type wallet-vs-invoice handling
4. **Credit/debit note generators** invoked by BIL-02-ADJ-01 Invoice Adjustments — same framework, linked to original invoice
5. **Bulk reversal first-class capability** with preview + dual approval + transactional execution + per-invoice failure audit
6. **Generation failure queue** with auto-retry, exponential backoff, give-up after max retries, admin review
7. **Universal idempotency** across all generators and all triggers (Kafka, scheduler, admin, ADJ-01 callout)
8. **Customer snapshot capture** at generation time per the immutability principle
9. **Grouping policy applied per generator** — every generator respects the operator's configured dimension and groups
10. **Line item builder** as a shared service producing two-level (SUMMARY + DETAIL) structure with per-line tax breakdown from PLM-CFG-02